

第五章 DSP的时钟与系统控制

- 5.1 概述
- 5.2 时钟和系统控制单元
- 5.3 OSC与PLL模块
- 5.4 低功耗模式模块
- 5.5 CPU看门狗模块
- 5.6 CPU 定时器
- 5.7 EALLOW保护的寄存器

5.1 概述

- F28027的系统控制单元包括时钟模块、低功耗模式模块和看门狗模块等。
- 当DSP上电并完成用户程序的装载后，应在系统主程序中对这些模块进行初始化，为系统程序的运行配置合适的运行环境。
- 同时F28027有三个32位的CPU定时器：
CPU定时器0和CPU定时器1可被应用系统使用，
CPU定时器2为DSP/BIOS预留，但如果不应用于DSP/BIOS，它也可被应用系统使用。
CPU定时器结构简单、使用方便，可以作为应用系统的定时时钟。

5.2 时钟和系统控制单元

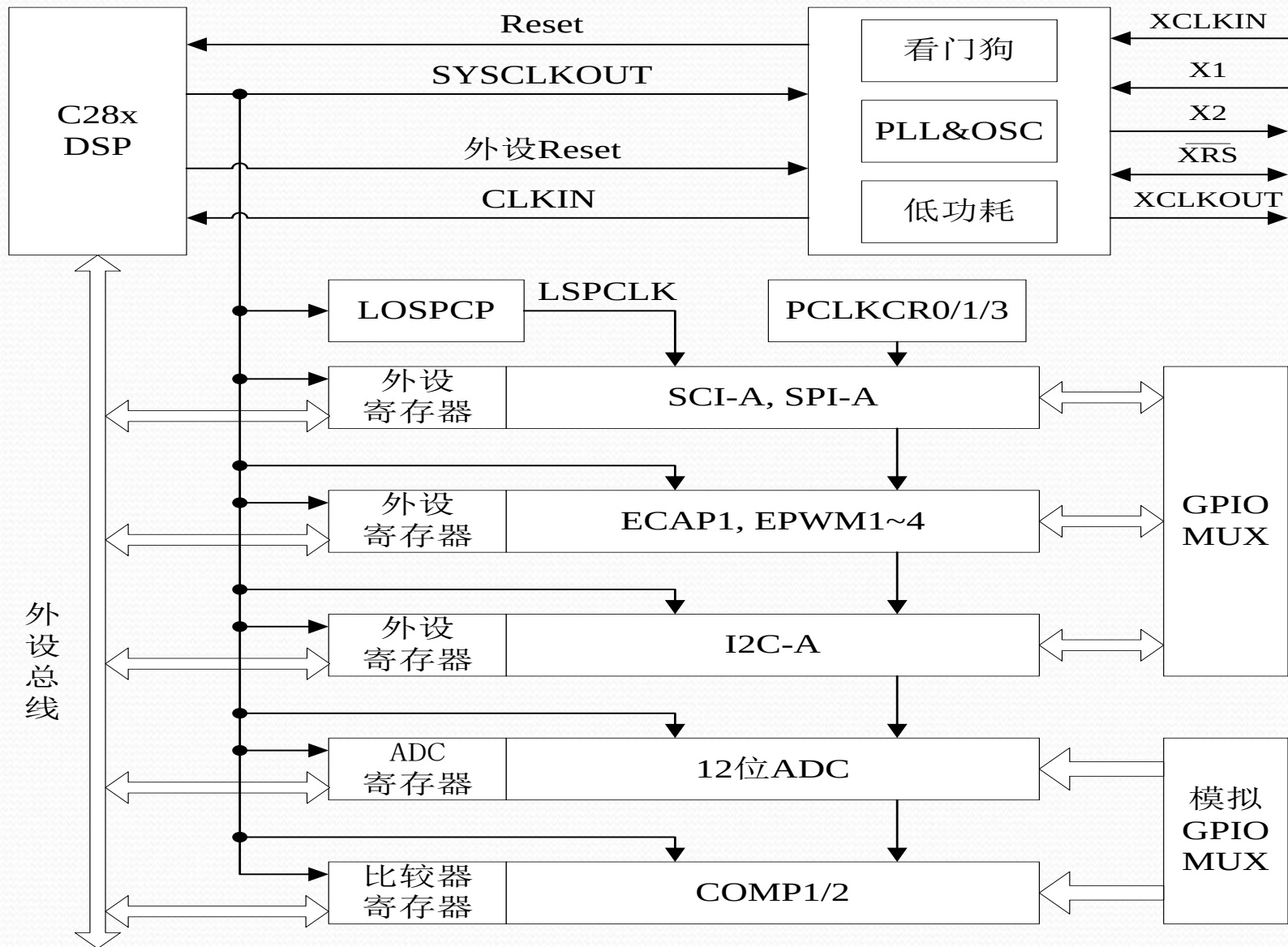


表5-1 时钟、PLL、OSC和低功耗模式寄存器

寄存器名	地址	大小(×16)	说明
XCLK	0x0000 7010	1	XCLKOUT/XCLKIN控制
PLLSTS	0x0000 7011	1	PLL状态寄存器
CLKCTL	0x0000 7012	1	时钟控制寄存器
PLLLOCKPRD	0x0000 7013	1	PLL锁定周期寄存器
INTOSC1TRIM	0x0000 7014	1	内部振荡器0调节寄存器
INTOSC2TRIM	0x0000 7016	1	内部振荡器1调节寄存器
LOSPCP	0x0000 701B	1	低速外设时钟预分频寄存器
PCLKCR0	0x0000 701C	1	外设时钟控制寄存器0
PCLKCR1	0x0000 701D	1	外设时钟控制寄存器1
LPMCR0	0x0000 701E	1	低功耗模式控制寄存器0
PCLKCR3	0x0000 7020	1	外设时钟控制寄存器3
PLLCR	0x0000 7021	1	PLL控制寄存器
SCSR	0x0000 7022	1	系统控制和状态寄存器
WDCNTR	0x0000 7023	1	看门狗计数器寄存器
WDKEY	0x0000 7025	1	看门狗复位关键字寄存器
WDCR	0x0000 7029	1	看门狗控制寄存器

- **PCLKCR0/1/3寄存器使能/禁能各种外设模块的时钟。**
- **从写PCLKCR0/1/3寄存器出现到操作有效之间有2个SYSCLKOUT周期的延迟。在尝试访问外设配置寄存器之前，必须考虑到这个延时。**
- **由于外设的GPIO在引脚上的复用，所有外设不能同时使用。**
- **虽然可以同时开启所有外设的时钟，但这样的配置可能用处不大。如果同时开启所有外设的时钟，消耗的电流会比要求的大。为了避免出现这种情况，只启用应用需要使用的时钟。**



PCLKCR0 外设时钟控制寄存器

位	域	值	描述 ⁽¹⁾
15-11	保留		
10	SCIAENCLK	0 1	SCI-A时钟使能。 不给SCI-A模块计时。（默认） ⁽¹⁾ 由低速时钟（LSPCLK）为SCI-A模块计时。
9	保留		
8	SPIAENCLK	0 1	SPI-A时钟使能。 不给SPI-A模块计时。（默认） ⁽¹⁾ 由低速时钟（LSPCLK）为SPI-A模块计时。
7-5	保留		
4	I2CAENCLK	0 1	I时钟使能。 不给I模块计时。（默认） ⁽¹⁾ 给I模块计时。
3	ADCENCLK	0 1	ADC时钟使能。 不给ADC模块计时。（默认） ⁽¹⁾ 给ADC模块计时。

PCLKCR0 外设时钟控制寄存器

位	域	值	描述 ⁽¹⁾
2	TBCLKSYNC	0	ePWM模块时基时钟（TBCLK）同步：允许用户将所有使能的ePWM模块与时基时钟（TBCLK）同步； 停止每个已使能ePWM模块内的TBCLK（时基时钟）（默认）。但是，如果PCLKCR1寄存器的ePWM时钟使能位被置位，那么ePWM模块仍然由SYSCLKOUT来计时，即使TBCLKSYNC为0。
		1	所有已使能的ePWM模块的时钟在已校准的TBCLK的第一个上升沿启动。为了得到完美同步的TBCLK，每个ePWM模块的TBCTL寄存器中的预分频器位必须设置相同。使能ePWM时钟的正确过程如下所示： ●在PCLKCR1寄存器中使能ePWM模块的时钟。 ●将TBCLKSYNC设置为0。 ●配置预分频器值和ePWM模式。 ●将TBCLKSYNC设置为1。
1	保留		保留
0	HRPWMENCLK	0	HRPWM时钟使能。
		0	HRPWM不使能。
		1	HRPWM使能。

PCLKCR1 外设时钟控制寄存器

位	域	值	描述 ⁽¹⁾
15-9	保留		
8	ECAP1ENCLK	0 1	eCAP1时钟使能。 不给eCAP1模块计时。（默认） ⁽²⁾ 由系统时钟（SYSCLKOUT）为eCAP1模块计时。
7-4	保留		
3	EPWM4ENCLK	0 1	ePWM4时钟使能。 ⁽³⁾ 不给ePWM4模块计时。（默认） ⁽²⁾ 由系统时钟（SYSCLKOUT）为ePWM4模块计时。
2	EPWM3ENCLK	0 1	ePWM3时钟使能。 ⁽³⁾ 不给ePWM3模块计时。（默认） ⁽²⁾ 由系统时钟（SYSCLKOUT）为ePWM3模块计时。
1	EPWM2ENCLK	0 1	ePWM2时钟使能。 ⁽³⁾ 不给ePWM2模块计时。（默认） ⁽²⁾ 由系统时钟（SYSCLKOUT）为ePWM2模块计时。
0	EPWM1ENCLK	0 1	ePWM1时钟使能。 ⁽³⁾ 不给ePWM1模块计时。（默认） ⁽²⁾ 由系统时钟（SYSCLKOUT）为ePWM1模块计时。

这个寄存器受EALLOW保护，为了启动ePWM模块内的ePWM时基时钟，还必须置位PCLKCR0的TBCLKSYNC位。

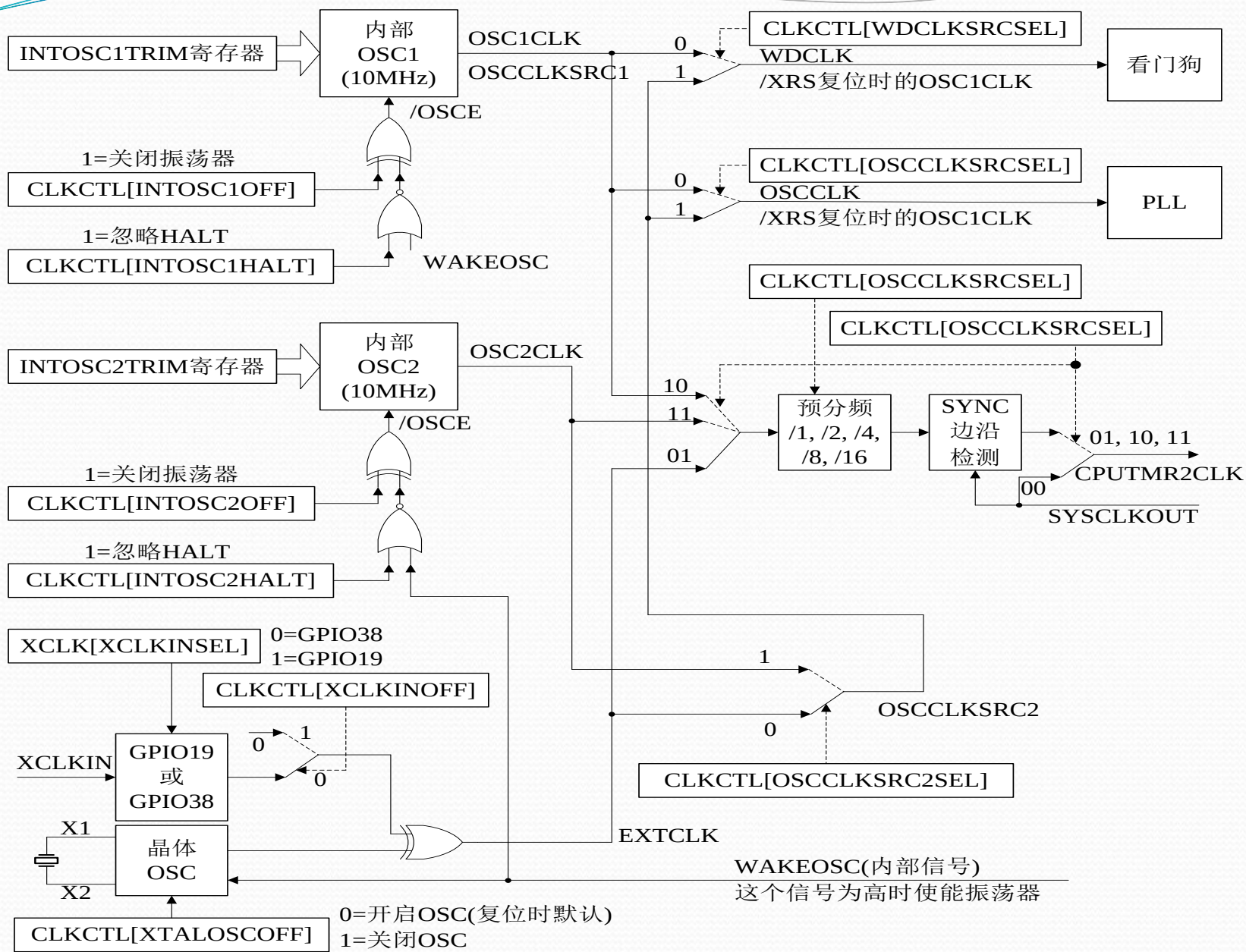
PCLKCR3 外设时钟控制寄存器

位	域	值	描述
15-14	保留		保留
13	GPIODINENCLK	0 1	GPIOD输入时钟使能。 不给GPIOD模块计时。 给GPIOD模块计时。
12-11	保留		保留
10	CPUTIMER2EN CLK	0 1	CPU定时器2时钟使能。 不给CPU定时器2计时。 给CPU定时器2计时。
9	CPUTIMER1EN CLK	0 1	CPU定时器1时钟使能。 不给CPU定时器1计时。 给CPU定时器1计时。
8	CPUTIMER0EN CLK	0 1	CPU定时器0时钟使能。 不给CPU定时器0计时。 给CPU定时器0计时。
7:2	保留		保留
1	COMP2ENCLK	0 1	COMP2时钟使能。 不给COMP2计时。 给COMP2计时。
0	COMP1ENCLK	0 1	COMP1时钟使能。 不给COMP1计时。 给COMP1计时。

LOSPCP 配置低速外设时钟预分频器

位	域	值	描述 ⁽¹⁾
15-3	保留		保留
2-0	LSPCLK		这些位用来配置低速外设时钟（LSPCLK）速率（相对于SYSCLKOUT）： 如果 $LOSPCP^{(2)} \neq 0$ ，则 $LSPCLK = SYSCLKOUT / (LOSPCP \times 2)$ ； 如果 $LOSPCP = 0$ ，则 $LSPCLK = SYSCLKOUT$ ；
		000	低速时钟 = $SYSCLKOUT / 1$
		001	低速时钟 = $SYSCLKOUT / 2$
		010	低速时钟 = $SYSCLKOUT / 4$ （复位默认值）
		011	低速时钟 = $SYSCLKOUT / 6$
		100	低速时钟 = $SYSCLKOUT / 8$
		101	低速时钟 = $SYSCLKOUT / 10$
		110	低速时钟 = $SYSCLKOUT / 12$
		111	低速时钟 = $SYSCLKOUT / 14$

5.3 OSC与PLL模块



输入时钟源的选择：

2802x器件有2个内部振荡器（INTOSC1和INTOSC2），不需要使用外部元件。这些器件还有一个片内基于PLL的时钟模块。

- 1. INTOSC1 —— 片内振荡器
- 2. INTOSC2 —— 片内振荡器
- 3. 无源晶振 —— X1/X2引脚连接外部晶振
- 4. 有源晶振 —— XCLKIN引脚连接时钟源

- **INTOSC1（内部零引脚振荡器1）**：这是片内内部振荡器1。它可以为看门狗模块、内核以及CPU Timer2提供时钟。
- **INTOSC2（内部零引脚振荡器2）**：这是片内内部振荡器2。它可以为看门狗模块、内核以及CPU Timer2提供时钟。INTOSC1和INTOSC2可以被单独选择用作看门狗模块、内核和CPU Timer2的时钟。
- **晶体/振荡器操作**：片内（晶体）振荡器允许使用一个外部晶体/振荡器连接到器件来提供时基（time base）。晶体/振荡器连接到X1/X2引脚。
- **外部时钟源操作**：如果未使用片内（晶体）振荡器，外部时钟源模式就允许绕过片内振荡器。这时，器件时钟通过XCLKIN引脚的外部时钟源输入产生。注意：**XCLKIN与GPIO19或GPIO38脚复用**。XCLKIN输入可以通过XCLK寄存器的XCLKINSEL位被选择用作GPIO19或GPIO38。

时钟源：

- | | |
|------------|------------|
| 1. INTOSC1 | 2. INTOSC2 |
| 3. 无源晶振 | 4. 有源晶振 |

为什么需要这么多时钟源？

如何配置所需要的时钟？

先选时钟源，再选PLL

体现在：根据需求配置相关的寄存器

时钟配置相关的寄存器

名称	地址	大小（×16）	描述
XCLK	0x0000-7010	1	XCLKOUT/XCLKIN控制
PLLSTS	0x0000-7011	1	PLL状态寄存器
CLKCTL	0x0000-7012	1	时钟控制寄存器
PLLLOCKPRD	0x0000-7013	1	PLL锁定周期寄存器
INTOSC1TRIM	0x0000-7014	1	内部振荡器1调整寄存器
INTOSC2TRIM	0x0000-7016	1	内部振荡器2调整寄存器
LOSPCP	0x0000-701B	1	低速外设时钟预分频器寄存器
PLLCR	0x0000-7021	1	PLL控制寄存器
SCSR	0x0000-7022	1	系统控制&状态寄存器

配置输入时钟源 & XCLKOUT 寄存器 (XCLK)

XCLK寄存器用来选择**XCLKIN**输入的**GPIO**引脚和配置**XCLKOUT**引脚的频率。

7	6	5	2	1	0
保留	XCLKINSEL	保留		XCLKOUTDIV	
R-0	R/W-1	R-0		R/W-0	

图例：RW=读/写；R=只读；-n=复位后的值

4

位	域	值	描述 ⁽¹⁾
15-7	保留		保留。
6	XCLKINSEL	0 1	XCLKIN源选择位：这个位选择XCLKIN输入源。 GPIO38是XCLKIN输入源（这也是JTAG端口的TCK源）。 GPIO19是XCLKIN输入源。
5-2	保留		保留
0-0	XCLKOUTDIV	00 01 10 11	XCLKOUT分频比：这两个位选择XCLKOUT频率相对于SYSCLKOUT的比率。比率是： XCLKOUT = SYSCLKOUT/4 XCLKOUT = SYSCLKOUT/2 XCLKOUT = SYSCLKOUT XCLKOUT = Off

配置器件时钟域 (CLKCTL)

CLKCTL寄存器用来选择时钟源，也用来配置时钟故障期间器件的行为

15	14	13	12	11	10	9	8
NMIRESETSEL	XTALOSCOFF	XCLKINOFF	WDHALTI	INTOSC2HALTI	INTOSC2OFF	INTOSC1HALTI	INTOSC1OFF
R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0
7	5	4	3	2	1	0	
TMR2CLKPRESCALE			TMR2CLKSRCSEL		WDCLKSRCSEL	OSCCLKSRC2SEL	OSCCLKSRCSEL
R/W-0			R/W-0		R/W-0	R/W-0	R/W-0

图例：RW=读/写；R=只读；-n=复位后的值

位	域	值	描述
15	NMIRESETSEL	0 1	NMI复位选择位：该位在检测到一个缺少时钟条件时直接产生 信号和使用 复位两者之间进行选择：没有任何延迟地直接驱动（复位时的默认状态）。 NMI看门狗复位（ ）启动。 注：不管作何选择，都会产生 信号。

位	域	值	描述
14	XTALOS COFF	0 1	晶体振荡器关闭位：如果晶体振荡器不使用，可以使用该位将其关闭。 晶体振荡器开启（复位时的默认状态）。 晶体振荡器关闭。
13	XCLKIN OFF	0 1	XCLKIN关闭位：该位关闭外部XCLKIN振荡器输入 XCLKIN振荡器输入启用（复位时的默认状态）。 XCLKIN振荡器输入关闭。 注：你需要通过XCLK寄存器的XCLKINSEL位选择XCLKIN 的GPIO引脚。更详细的信息请见XCLK寄存器的描述。如果使用XCLKIN，XTALOSCOFF就必须置位。
12	WDHALT I	0 1	看门狗停机模式忽略位：该位选择是否通过停机模式自动开启/关闭看门狗。这个特性可以在停机模式有效时用来允许所选的WDCLK源继续计时看门狗。这可以允许看门狗周期性地唤醒器件。 看门狗自动通过停机模式开启/关闭（复位时的默认状态）。 看门狗忽略停机模式。
11	INTOSC2 HALTI	0 1	内部振荡器2 停机模式忽略位：该位选择是否通过停机模式自动开启/关闭内部振荡器2。这个特性可以在停机模式有效时用来允许内部振荡器继续计时。这将使能器件更快地从停机模式中唤醒。 内部振荡器2自动通过停机模式开启/关闭（复位时的默认状态）。 内部振荡器2忽略停机模式。
10	INTOSC2 OFF	0 1	内部振荡器2关闭位：该位关闭振荡器2 内部振荡器2开启（复位时的默认状态）。 内部振荡器2关闭。如果内部振荡器2不被使用，用户可以使用该位将其关闭。这个选择不受缺少时钟检测电路影响。
9	INTOSC1 HALTI	0 1	内部振荡器1 停机模式忽略位：该位选择是否通过停机模式自动开启/关闭内部振荡器1。 内部振荡器1自动通过停机模式开启/关闭（复位时的默认状态）。 内部振荡器1忽略停机模式。这个特性可以在停机模式有效时用来允许内部振荡器继续计时。这将使能器件更快地从停机模式中唤醒。

位	域	值	描述
8	INTOSC1OFF	0 1	内部振荡器1关闭位：该位关闭振荡器1。 内部振荡器1开启（复位时的默认状态）。 内部振荡器1关闭。如果内部振荡器1不被使用，用户可以使用该位将其关闭。这个选择不受缺少时钟检测电路影响。
7-5	TMR2CLKPRESCALE	000 001 010 011 100 101 110 111	CPU Timer2时钟预分频值：这些位为所选的CPU Timer2时钟源选择预分频值。这个选择不受缺少时钟检测电路影响。 /1（复位时的默认状态） /2 /4 /8 /16 保留 保留 保留
4-3	TMR2CLKSRCSEL	00 01 10 11	CPU Timer2时钟源选择位：该位为CPU Timer2选择时钟源选择SYSCLKOUT（复位时的默认状态，绕过预分频器）。 选择外部振荡器（XOR输出）。 选择内部振荡器1。 选择内部振荡器2。这个选择不受缺少时钟检测电路影响。
2	WDCLKSRCSEL	0 1	看门狗时钟源选择位：该位选择看门狗的时钟源。在 为低时和 变为高之后，默认选择内部振荡器1。用户需要在初始化过程中选择外部振荡器或内部振荡器2。如果缺少时钟检测电路检测到缺少时钟，那么该位被强制为0，并选择内部振荡器1。用户更改该位不影响PLLCR值。 选择内部振荡器1（复位时的默认状态）。 选择外部振荡器或内部振荡器2。

位	域	值	描述
1	OSCCLKSRC2SEL	0 1	振荡器2时钟源选择位：这个位用来在内部振荡器2或外部振荡器两者之间作选择。这个选择不受缺少时钟检测电路影响。 选择外部振荡器（复位时的默认状态）。 选择内部振荡器2。
0	OSCCLKSRCSEL	0 1	振荡器时钟源选择位。这个位选择振荡器的时钟源。在 为低时和 变为高之后，默认选择内部振荡器1。用户需要在初始化过程中选择外部振荡器或内部振荡器2。只要用户使用这些位来改变时钟源，PLLCR寄存器将被自动强制为零。这阻止了潜在的PLL过冲。然后，用户必须写PLLCR寄存器来配置合适的分频比。如果必要，用户也可以使用PLLLOCKPRD寄存器来配置PLL锁定周期以缩短锁定时间。如果缺少时钟检测电路检测到缺少时钟，那么这个位被自动强制为0，并选择内部振荡器1作为时钟。PLLCR寄存器也将自动强制为0来防止所有潜在的过冲。 选择内部振荡器1（复位时的默认状态）。 选择外部振荡器或内部振荡器2。注：如果用户希望使用振荡器2或外部振荡器来计时CPU，必须应该先配置这个位，然后再写OSCCLKSRCSEL位。

示例一：在没有外部时钟的情况下切换到 INTOC2

为了使器件能够正常工作，应用代码需要：

先向CLKCTL寄存器的XTALOSCOFF和XCLKINOFF位写入1。这样来告知时钟切换电路没有外部时钟。

只有这么做之后，OSCCLKSRCSEL和OSCCLKSRC2SEL位才能被写入。

```
SysCtrlRegs.CLKCTL.all = 0x6000; // Set XTALOSCOFF=1 & XCLKINOFF=1  
SysCtrlRegs.CLKCTL.all = 0x6003; // Set OSCCLKSRCSEL=1 & OSCCLKSRC2SEL=1
```

写的序列分为下列两步：

1. 将XTALOSCOFF & XCLKINOFF 位置 1
2. 将OSCCLKSRCSEL & OSCCLKSRC2SEL 位置 1

示例二：选择内部振荡器1

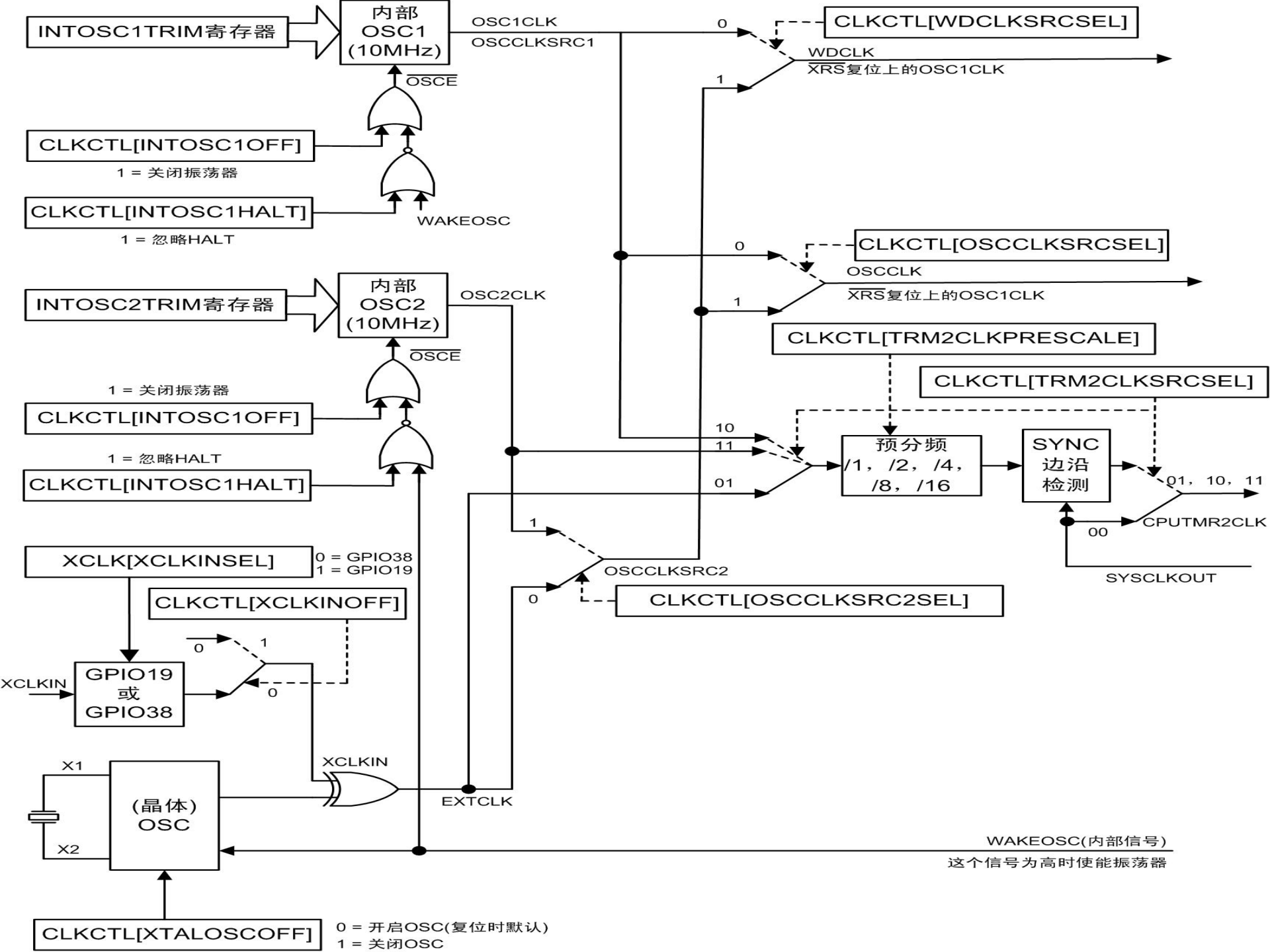
```
void IntOsc1Sel (void) {  
    EALLOW;  
    SysCtrlRegs.CLKCTL.bit.XCLKINOFF=1;      // Turn off XCLKIN  
    SysCtrlRegs.CLKCTL.bit.XTALOSCOFF=1;      // Turn off XTALOSC  
    SysCtrlRegs.CLKCTL.bit.INTOSC2OFF=1;      // Turn off INTOSC2  
    SysCtrlRegs.CLKCTL.bit.INTOSC1OFF = 0;  
    SysCtrlRegs.CLKCTL.bit.OSCCLKSRCSEL=0;    // Clk Src = INTOSC1  
    EDIS;  
}
```


示例三：选择XTAL振荡器

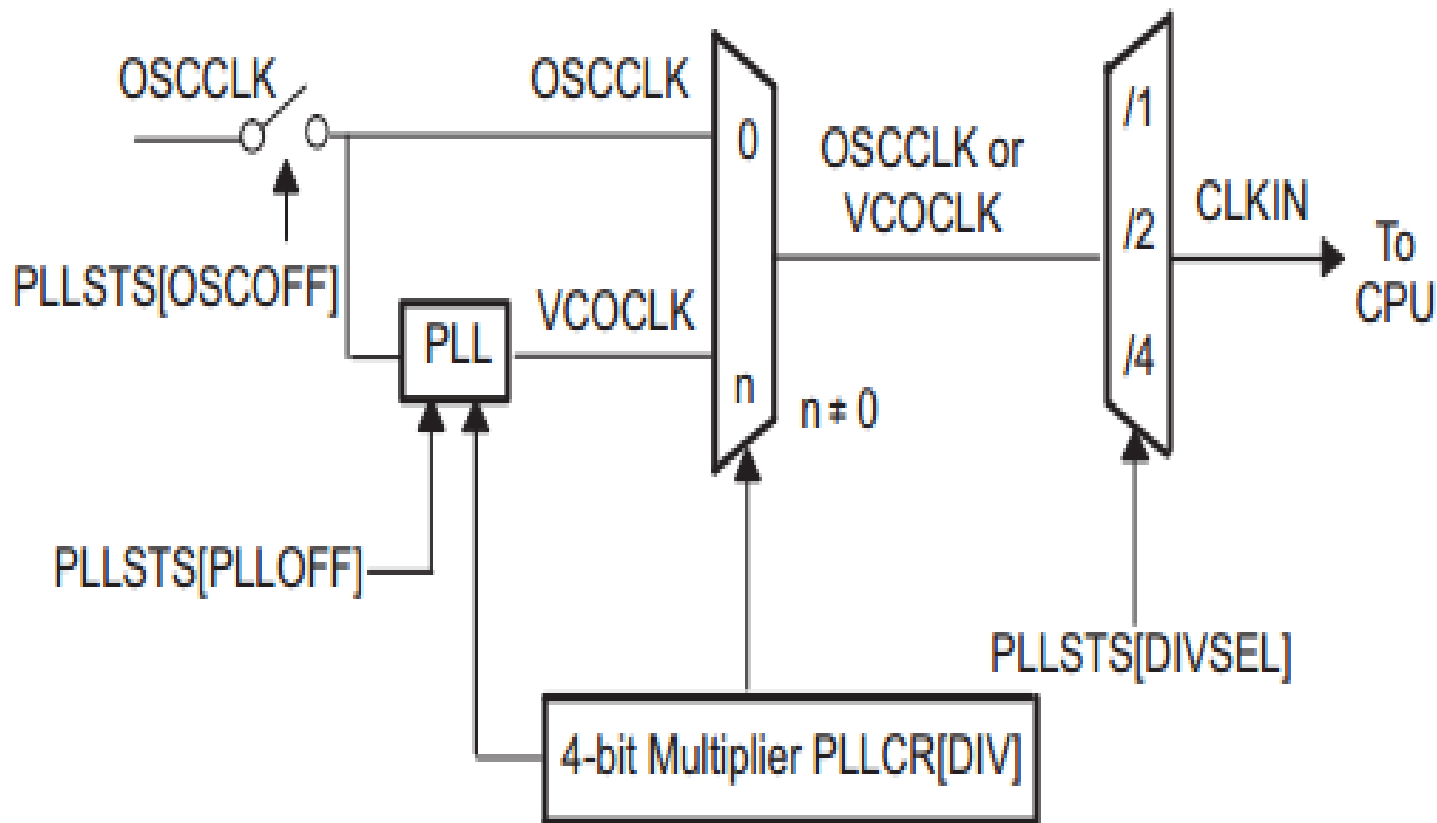
```
void XtalOscSel(void) {  
    EALLOW;  
    SysCtrlRegs.CLKCTL.bit.INTOSC2OFF = 1;      // Turn off INTOSC2  
    SysCtrlRegs.CLKCTL.bit.INTOSC1OFF = 1;      // Turn off INTOSC1  
    SysCtrlRegs.CLKCTL.bit.XCLKINOFF = 1;        // Turn off XCLKIN  
    SysCtrlRegs.CLKCTL.bit.XTALOSCOFF = 0;       // Turn on XTALOSC  
  
    SysCtrlRegs.CLKCTL.bit.OSCCLKSRC2SEL = 0;    // Switch to external clock  
    SysCtrlRegs.CLKCTL.bit.OSCCLKSRCSEL = 1;  
                                                // Switch from INTOSC1 to INTOSC2/ext clk  
    EDIS;  
}
```

示例四：选择外部时钟

```
void ExtOscSel (void) {  
    EALLOW;  
    SysCtrlRegs.XCLK.bit.XCLKINSEL = 1;    // 1-GPIO19 = XCLKIN, 0-GPIO38 = XCLKIN  
    SysCtrlRegs.CLKCTL.bit.INTOSC2OFF = 1; // Turn off INTOSC2  
    SysCtrlRegs.CLKCTL.bit.INTOSC1OFF = 1; // Turn off INTOSC1  
    SysCtrlRegs.CLKCTL.bit.XTALOSCOFF = 1; // Turn off XTALOSC  
    SysCtrlRegs.CLKCTL.bit.XCLKINOFF = 0;  // Turn on XCLKIN  
    SysCtrlRegs.CLKCTL.bit.OSCCLKSRC2SEL = 0; // Switch to external clock  
    SysCtrlRegs.CLKCTL.bit.OSCCLKSRCSEL = 1; // Switch from INTOSC1 to INTOSC2/ext clk  
    EDIS;  
}
```

基于PLL的时钟模块



选择的输入时钟和PLLCR[DIV]位应该使PLL(VCOCLK)的输出频率是最小值50MHz。

PLL状态寄存器 (PLLSTS)

15		14				9		8							
NORMRDYE		保留						DIVSEL							
R-0						R/W-0									
7		6		5		4		3		2		1		0	
DIVSEL		MCLKOFF		OSCOFF		MCLKCLR		MCLKSTS		PLLOFF		保留		PLLLOCKS	
R/W-0		R/W-0		R/W-0		R/W-0		R-0		R/W-0		R-0		R-1	

图例：RW=读/写；R=只读；-n=复位后的值

PLL控制寄存器 (PLLCR)

15			4	3	0
保留			DIV		
R-0			RW-0		

图例：RW=读/写；R=只读；-n=复位后的值

选择的输入时钟和PLLCR[DIV]位应该使PLL(VCOCLK)的输出频率是最小值50MHz。

PLL 控制寄存器 (PLLCR)

PLLCR[DIV] Value ⁽³⁾	SYSCLKOUT (CLKIN) ⁽²⁾		
	PLLSTS[DIVSEL] = 0 or 1	PLLSTS[DIVSEL] = 2	PLLSTS[DIVSEL] = 3
0000 (PLL bypass)	OSCCLK/4 (Default)	OSCCLK/2	OSCCLK/1
0001	$(OSCCLK * 1)/4$	$(OSCCLK * 1)/2$	$(OSCCLK * 1)/1$
0010	$(OSCCLK * 2)/4$	$(OSCCLK * 2)/2$	$(OSCCLK * 2)/1$
0011	$(OSCCLK * 3)/4$	$(OSCCLK * 3)/2$	$(OSCCLK * 3)/1$
0100	$(OSCCLK * 4)/4$	$(OSCCLK * 4)/2$	$(OSCCLK * 4)/1$
0101	$(OSCCLK * 5)/4$	$(OSCCLK * 5)/2$	$(OSCCLK * 5)/1$
0110	$(OSCCLK * 6)/4$	$(OSCCLK * 6)/2$	$(OSCCLK * 6)/1$
0111	$(OSCCLK * 7)/4$	$(OSCCLK * 7)/2$	$(OSCCLK * 7)/1$
1000	$(OSCCLK * 8)/4$	$(OSCCLK * 8)/2$	$(OSCCLK * 8)/1$
1001	$(OSCCLK * 9)/4$	$(OSCCLK * 9)/2$	$(OSCCLK * 9)/1$
1010	$(OSCCLK * 10)/4$	$(OSCCLK * 10)/2$	$(OSCCLK * 10)/1$
1011	$(OSCCLK * 11)/4$	$(OSCCLK * 11)/2$	$(OSCCLK * 11)/1$
1100	$(OSCCLK * 12)/4$	$(OSCCLK * 12)/2$	$(OSCCLK * 12)/1$
1101-1111	Reserved	Reserved	Reserved

PLL的配置步骤:

```
void InitPll(Uint16 val, Uint16 divsel)
{
    volatile Uint16 iVol;
    // Make sure the PLL is not running in limp mode
    // 第一步
    if (SysCtrlRegs.PLLSTS.bit.MCLKSTS != 0){
        EALLOW;
        // OSCCLKSRC1 failure detected. PLL running in limp mode.
        // Re-enable missing clock logic.
        SysCtrlRegs.PLLSTS.bit.MCLKCLR = 1;
        EDIS;
        // Replace this line with a call to an appropriate
        // SystemShutdown(); function.
        asm("      ESTOP0"); // Uncomment for debugging purposes
    }
```

// 第二步

// DIVSEL MUST be 0 before PLLCR can be changed from
// 0x0000. It is set to 0 by an external reset XRSn
// This puts us in ¼

```
if (SysCtrlRegs.PLLSTS.bit.DIVSEL != 0){  
    EALLOW;  
    SysCtrlRegs.PLLSTS.bit.DIVSEL = 0;  
    EDIS;  
}
```


// 第三步:

// Change the PLLCR

```
if (SysCtrlRegs.PLLCR.bit.DIV != val){  
    EALLOW;  
    SysCtrlRegs.PLLSTS.bit.MCLKOFF = 1;  
    SysCtrlRegs.PLLCR.bit.DIV = val;  
    EDIS;  
    DisableDog();  
    while(SysCtrlRegs.PLLSTS.bit.PLLLOCKS != 1){  
        // Uncomment to service the watchdog  
        // ServiceDog();  
    }  
    EALLOW;  
    SysCtrlRegs.PLLSTS.bit.MCLKOFF = 0;  
    EDIS;  
}
```

// 第四步:

// If switching to 1/2

```
if((divsel == 1)|| (divsel == 2)){
```

```
    EALLOW;
```

```
    SysCtrlRegs.PLLSTS.bit.DIVSEL = divsel;
```

```
    EDIS;
```

```
}
```

// If switching to 1/1

// * First go to 1/2 and let the power settle

// The time required will depend on the system, this is only an example

// * Then switch to 1/1

```
if(divsel == 3){
```

```
    EALLOW;
```

```
    SysCtrlRegs.PLLSTS.bit.DIVSEL = 2;
```

```
    DELAY_US(50L);
```

```
    SysCtrlRegs.PLLSTS.bit.DIVSEL = 3;
```

```
    EDIS;
```

```
}
```

```
}
```


● 5.4 低功耗模式模块

低功耗模式由LPMCR0寄存器来控制：

15	14	8	7	2	1	0
WDINTE	Reserved			QUALSTDBY		LPM
R/W-0	R-0			R/W-0x3F		R/W-0

低功耗模式之**IDLE模式**：
CPU进入低功耗模式，

此时OSC和PLL模块的时钟并不关闭，但**外设时钟可以单独关闭**，这样在IDLE期间需要运行的外设可以继续保持运行。

一个被允许的**外设中断或看门狗定时器**可以使CPU退出IDLE模式。

低功耗模式之**STANDBY**模式：

CPU和外设的时钟**全部关闭**，但OSC和PLL模块继续工作。
外部中断事件将使CPU退出STANDBY模式，检测到外部中断事件下一个有效周期外设开始工作。

低功耗模式之**HALT**模式：

F28027芯片被关闭，处于**最低功耗模式**下。
复位或外部信号或看门狗将使CPU退出HALT模式。

表5-5 低功耗模式总结

模式	LPMCRo(1:0)	OSCCLK	CLKIN	SYSCLKOUT	退出
IDLE	00	on	on	On	$\overline{XRS}$ 、看门狗中断和任何使能中断
STANDBY	01	on(CPU看门狗仍运行)	on	Off	$\overline{XRS}$ 、看门狗中断、GPIO-A信号以及调试器
HALT	1X	Off(片内晶体振荡器和PLL关闭，片内振荡器和看门狗状态取决于用户代码)	off	off	$\overline{XRS}$ 、看门狗中断、GPIO-A信号以及调试器

● 5.5 CPU看门狗模块

5.5.1 看门狗工作原理

看门狗实际上就是一个计数器，

它不依赖CPU运行，

主要用来复位芯片的软硬件故障。

一旦芯片出现故障，看门狗计数器就不能按时被清零，计数器就会产生溢出，则将触发复位或中断。

F28027看门狗模块内含一个8位的加计数器。

只要8位的看门狗递增计数器到达最大值，看门狗模块就产生一个512个振荡器时钟宽的输出脉冲。

为了阻止这种情况发生：

用户可以禁能计数器，

或者，软件周期性地向看门狗密钥寄存器写入一个**0x55+0xAA**序列，使看门狗计数器复位。

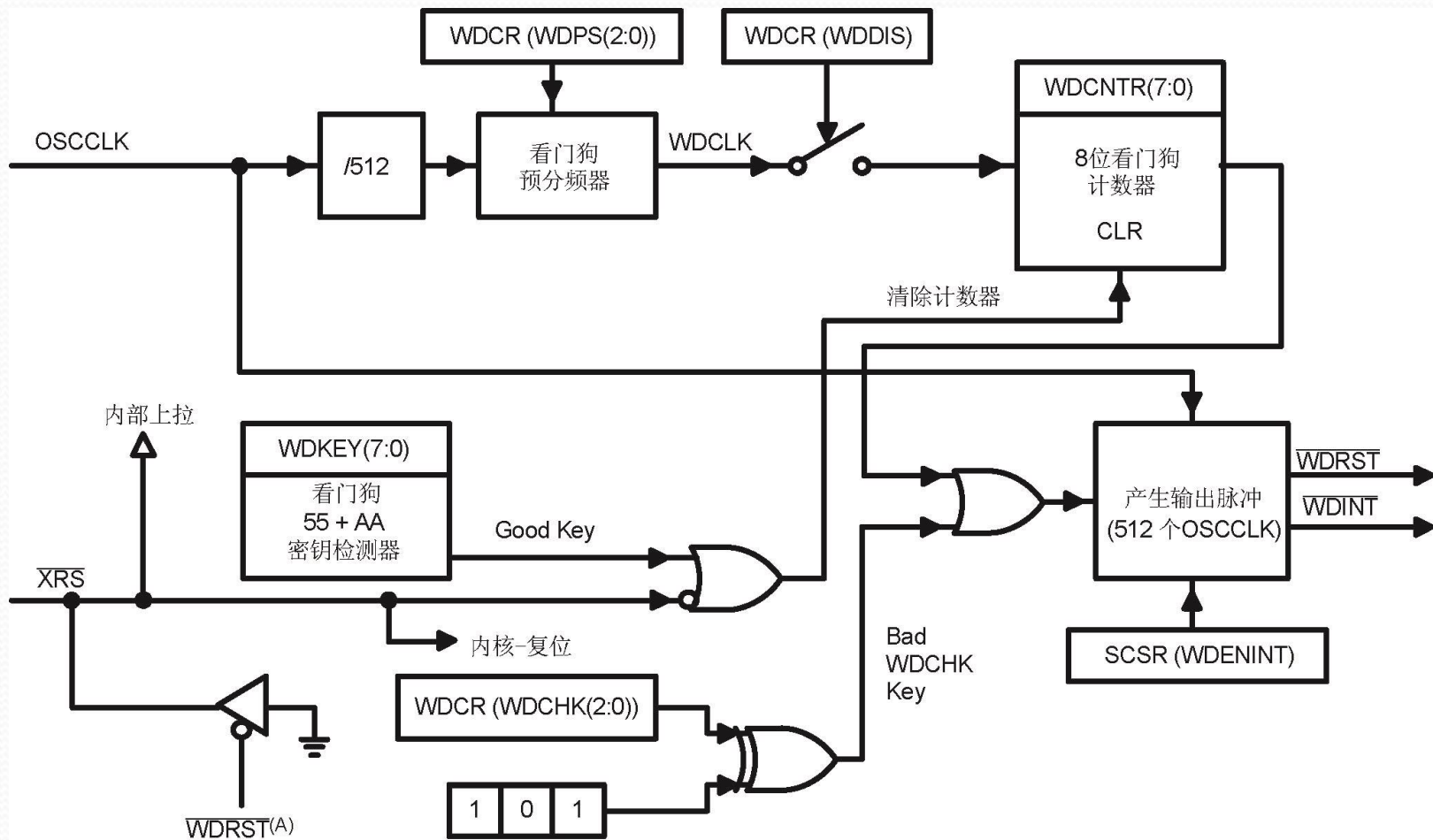
当8位的看门狗计数器（WDCNTR）溢出前正确的序列被写入WDKEY寄存器时，WDCNTR被复位。

值0x55被写入WDKEY时允许复位WDCNTR。

当写入WDKEY寄存器的下个值是0xAA时，WDCNTR被复位。

除0x55或0xAA以外的其它值写入WDKEY不会产生任何操作。
并不是将0x55和0xAA以任何顺序写入WDKEY都会造成复位。

Watchdog 模块框图



注：当出现看门狗复位时， $\overline{\text{WDRST}}$ 和 $\overline{\text{XRS}}$ 信号被驱动为低电平 512 个 OSCCLK 周期。同样地，如果看

门狗中断被使能，当中断发生时， $\overline{\text{WDINT}}$ 信号被驱动为低电平 512 个 OSCCLK 周期。

Watchdog 相关寄存器 (SCSR)

7	3	2	1	0	
保留			WDINTS	WDENINT	WDOVERRIDE
R-0			R-1	R/W-0	R/W1C-1

位	域	值	描述(1)
15-3	保留		
2	WDINTS	 0 1	看门狗中断状态位。WDINTS 反映了看门狗模块 $\overline{\text{WDINT}}$ 信号的当前状态。WDINTS 与 $\overline{\text{WDINT}}$ 的状态保持一致,并追加 2 个 SYSCLKOUT 周期的状态保持时间。 如果看门狗中断用来将器件从空闲或待机低功率模式中唤醒,在尝试回到空闲或待机模式之前,使用该位来确保 $\overline{\text{WDINT}}$ 无效。 0 看门狗中断信号 ($\overline{\text{WDINT}}$) 有效。 1 看门狗中断信号 ($\overline{\text{WDINT}}$) 无效。
1	WDENINT	 0 1	看门狗中断使能。 0 看门狗复位 ($\overline{\text{WDRST}}$) 输出信号被使能,看门狗中断 ($\overline{\text{WDINT}}$) 输出信号被禁能。这是复位时 ($\overline{\text{XRS}}$) 的默认状态。当看门狗中断出现时, $\overline{\text{WDRST}}$ 信号将保持低电平 512 个 OSCCLK 周期。 如果 $\overline{\text{WDINT}}$ 为低时 WDENINT 位被清除,将立刻产生复位。可以读取 WDINTS 位来确定 $\overline{\text{WDINT}}$ 信号的状态。 1 $\overline{\text{WDRST}}$ 输出信号被禁能, $\overline{\text{WDINT}}$ 输出信号被使能。当看门狗中断出现时, $\overline{\text{WDINT}}$ 信号将保持低电平 512 个 OSCCLK 周期。 如果看门狗中断用来将器件从空闲或待机低功率模式中唤醒,在尝试回到空闲或待机模式前,使用 WDINTS 位来确保 $\overline{\text{WDINT}}$ 信号无效。

Watchdog 相关寄存器 (SCSR)



位 [↗]	域 [↗]	值 [↗]	描述 ^{(1)↗}
0 [↗]	WDOVERRIDE [↗]	↗ 0[↗] ↗ 1[↗]	看门狗忽略。[↗] 写 0 无影响。如果该位被清除，它将保持在此状态，直到产生复位。这个位的当前状态可由用户读出。[↗] 你可以更改看门狗控制（WDCR）寄存器中看门狗禁能位（WDDIS）的状态。如果 WDOVERRIDE 位通过写入 1 被清除，你就不能修改 WDDIS 位的值。[↗]

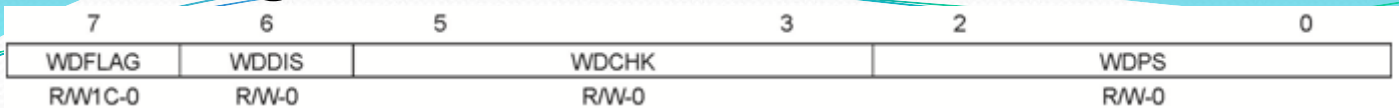
Watchdog 相关寄存器 (WDCNTR)

位	域	描述
15-8	保留	保留
7-0	WDCNTR	这些位包含看门狗计数器的当前值。8位计数器以看门狗时钟（WDCLK）速率持续递增。如果计数器溢出，看门狗就启动一次复位。如果WDKEY寄存器写入一个有效的值组合，计数器就被复位到零。看门狗时钟速率在WDCR寄存器中配置。

Watchdog 相关寄存器 (WDKEY)

位	域	值	描述 ⁽¹⁾
15-8	保留		保留
7-0	WDKEY	0x55+0xA A 其它值	不同WDKEY写序列的范例请参考表 2.22。 先后将0x55和0xAA写入WDKEY将WDCNTR位清除。 写入除0x55或0xAA以外的其它值不会产生任何动作。 如果写入0x55之后写入的不是0xAA，那么序列必须从写0x55这里重新开始。 读取WDKEY返回WDCR寄存器的值。

Watchdog 相关寄存器 (WDCR)



15-8	保留		保留
7	WDFLAG	0 1	看门狗复位状态标志位。 由 $\overline{\text{XRS}}$ 引脚或上电引起的复位。该位保持锁存，直到写入1将条件清除。写0被忽略。 指示一个看门狗复位 ($\overline{\text{WDRST}}$) 产生的复位条件。
6	WDDIS	0 1	看门狗禁能。复位时看门狗模块被使能。 使能看门狗模块。只有SCSR寄存器的WDOVERRIDE位被设置为1时WDDIS才能被修改。（默认） 禁能看门狗模块。
5-3	WDCHK	0,0,0 其它值	看门狗检查。 只要写这个寄存器，就必须向该域写入值1,0,1，除非你希望通过软件复位器件。如果看门狗被使能，则写入任何其它值都会造成器件立刻复位或看门狗中断被使用。这3个位读出时总是零（0,0,0）。这个特性可以用来产生一个DSP的软件复位。
2-0	WDPS	000 001 010 011 100 101 110 111	看门狗预分频。这些位配置看门狗计数器时钟（WDCLK）速率（相对于OSCCLK/512）： WDCLK = OSCCLK/512/1（默认） WDCLK = OSCCLK/512/1 WDCLK = OSCCLK/512/2 WDCLK = OSCCLK/512/4 WDCLK = OSCCLK/512/8 WDCLK = OSCCLK/512/16 WDCLK = OSCCLK/512/32 WDCLK = OSCCLK/512/64

Watchdog 计数器操作方式

1. 0X55 & 0XAA 序列的顺序写入
2. WDDIS 位的置一 (WDOVERRIDE = 1)

5.6 CPU定时器

■应该说定时器是微机系统**最重要的**外设，

■作用：定时（基本）

■ 计数（基本）

■ PWM（扩展）

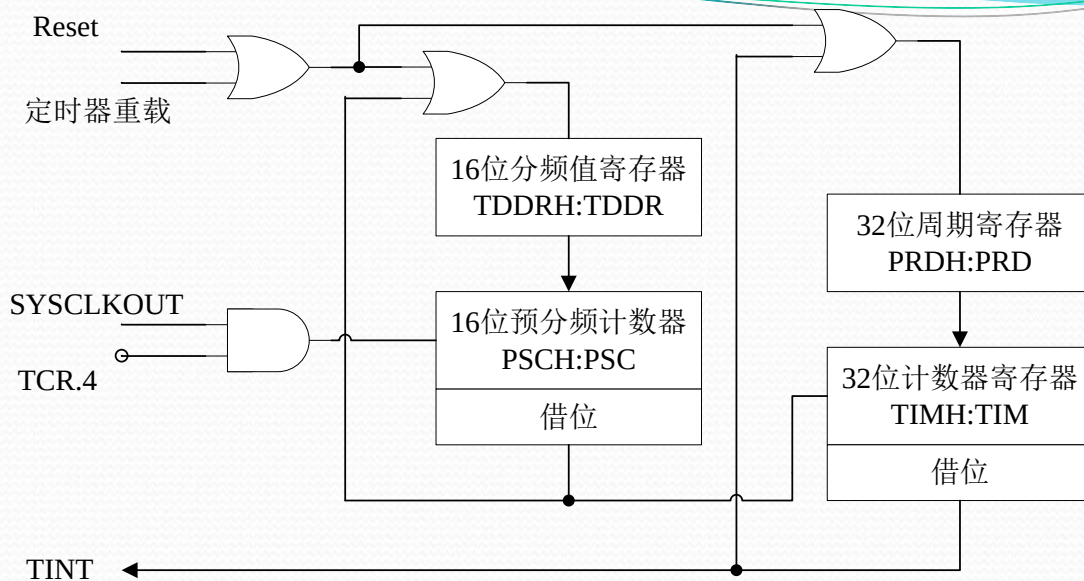
■ 捕获（扩展）

■ **ADC触发（扩展）**

■

■正是因为Timer太重要，F2802x把每种用途的Timer独立出来，

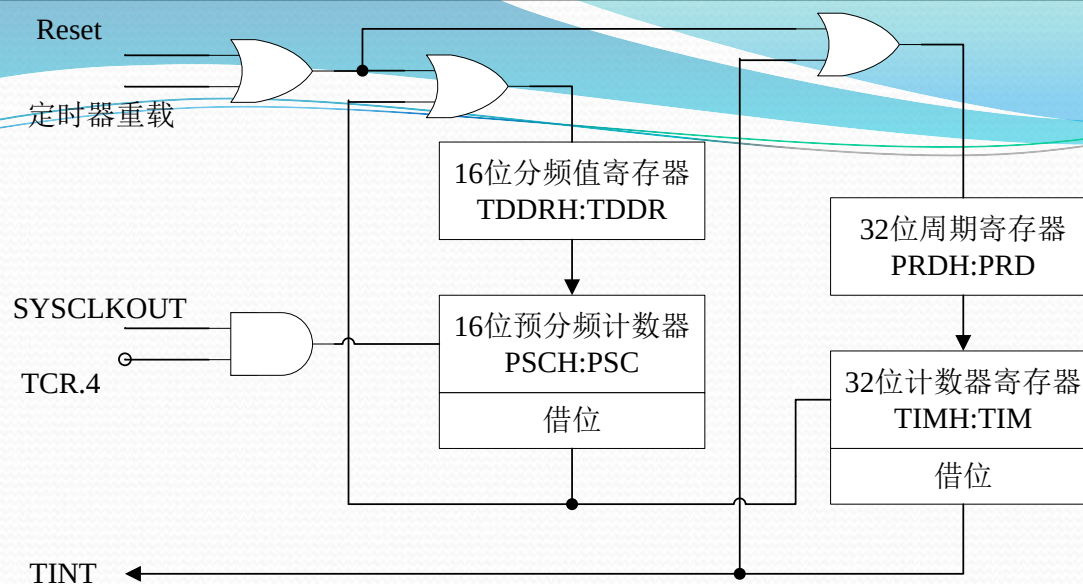
■本章的CpuTimer只是最基本的**定时器**



它主要由**16位的分频器**和**32位的计数器**构成。

分频器对系统时钟SYSCLKOUT进行分频，分频后的时钟作为32位计数器的工作时钟。

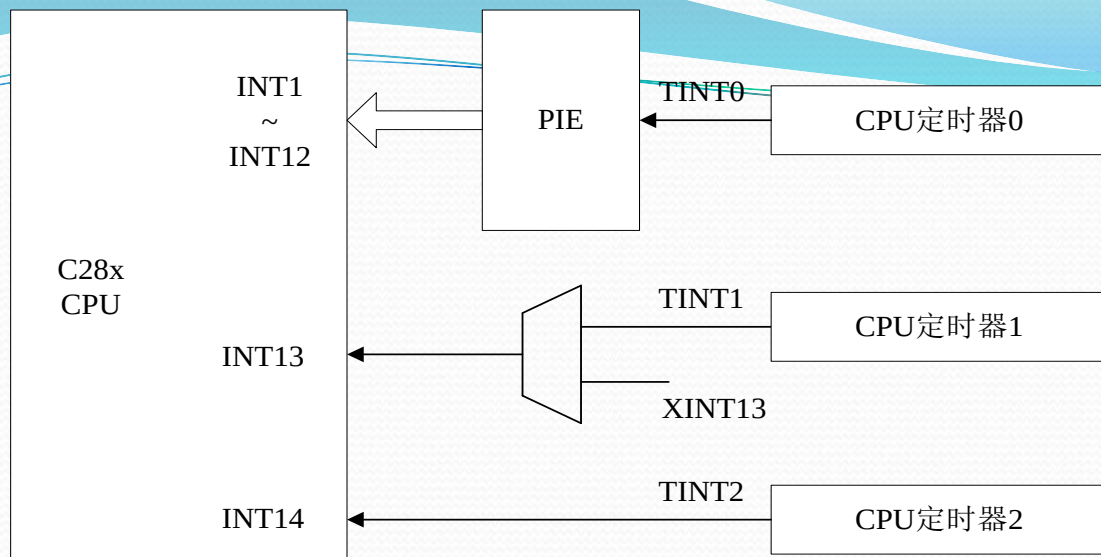
32位的计数器对工作时钟进行减计数，当计数器的计数值减为0后向CPU发出中断请求信号TINT。



16位的分频器和32位的计数器均由**两个寄存器**组成：
一个寄存器用来装载计数常数，另一个寄存器用作减计数器。

32位计数器：
装载计数常数的寄存器为定时器周期寄存器PRDH:PRD
用于减计数的寄存器为32位计数器TIMH:TIM。

16位分频器：
寄存器TDDRH:TDDR用于装载分频系数，
预分频计数器PSCH:PSC用于实现分频计数



3个定时器工作原理基本相同，但向CPU申请中断的途径不同：

定时器2的中断请求信号直接送到CPU的中断控制逻辑，

定时器1的中断申请要经过多路器的选择后才送到CPU的中断控制逻辑，

定时器0的中断申请要经过PIE模块处理才能送到CPU的中断控制逻辑。

CPU定时器学习和使用的核心操作：

如何根据定时器的工作原理和结构，配置定时长度和中断。

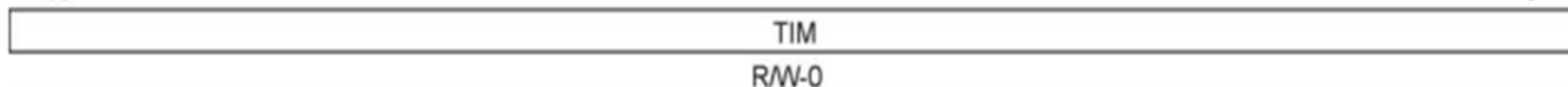
具体体现为：

- 1) 配置定时长度，包括PRD和TPR**
- 2) 配置控制寄存器，包括是否允许中断、TRB和TSS**

寄存器的使用！！

表5-7 定时器配置和控制寄存器

寄存器名	地址	大小(×16)	说明
TIMER0TIM	0x0C00	1	CPU定时器0计数器寄存器
TIMER0TIMH	0x0C01	1	CPU定时器0计数器高位寄存器
TIMER0PRD	0x0C02	1	CPU定时器0周期寄存器
TIMER0PRDH	0x0C03	1	CPU定时器0周期高位寄存器
TIMER0TCR	0x0C04	1	CPU定时器0控制寄存器
TIMER0TPR	0x0C06	1	CPU定时器0预分频寄存器
TIMER0TPRH	0x0C07	1	CPU定时器0预分频高位寄存器
TIMER1TIM	0x0C08	1	CPU定时器1计数器寄存器
TIMER1TIMH	0x0C09	1	CPU定时器1计数器高位寄存器
TIMER1PRD	0x0C0A	1	CPU定时器1周期寄存器
TIMER1PRDH	0x0C0B	1	CPU定时器1周期高位寄存器
TIMER1TCR	0x0C0C	1	CPU定时器1控制寄存器
TIMER1TPR	0x0C0E	1	CPU定时器1预分频寄存器
TIMER1TPRH	0x0C0F	1	CPU定时器1预分频高位寄存器
TIMER2TIM	0x0C10	1	CPU定时器2计数器寄存器
TIMER2TIMH	0x0C11	1	CPU定时器2计数器高位寄存器
TIMER2PRD	0x0C12	1	CPU定时器2周期寄存器
TIMER2PRDH	0x0C13	1	CPU定时器2周期高位寄存器
TIMER2TCR	0x0C14	1	CPU定时器2控制寄存器
TIMER2TPR	0x0C16	1	CPU定时器2预分频寄存器
TIMER2TPRH	0x0C17	1	CPU定时器2预分频高位寄存器



图例：R/W=读/写；R=只读；-n=复位后的值

TIMERxTIM 寄存器 (x=1, 2, 3)

位	域	描述
15-0	TIM	CPU 定时器计数器寄存器 (TIMH:TIM): TIM 寄存器保存定时器当前 32 位计数值的低 16 位。TIMH 保存定时器当前 32 位计数值的高 16 位。TIMH: TIM 每 (TDDRH:TDDR+1) 个时钟周期递减一次, 这里的 TDDRH:TDDR 是预分频 divide-down 值。当 TIMH:TIM 递减到零时, TIMH:TIM 寄存器装入 PRDH:PRD 寄存器包含的周期值。产生定时器中断 (TINT) 信号。

32位 CPU 计数器寄存器 (TIMERxTIM)

TIMERxTIMH与之类似

PRD

R/W-0

图例：R/W=读/写；R=只读；-n=复位后的值

TIMERxPRD 寄存器 (x=1, 2, 3)

TIMERxPRD 寄存器域描述

位	域	描述
15-0	PRD	CPU 定时器周期寄存器 (PRDH:PRD)：PRD 寄存器保存 32 位周期值的低 16 位。PRDH 寄存器保存 32 位周期值的高 16 位。当 TIMH:TIM 递减到零时，在下个定时器输入时钟周期（预分频器的输出）开始时 TIMH:TIM 寄存器装入 PRDH:PRD 寄存器包含的周期值。当定时器控制寄存器 (TCR) 的定时器重装位 (TRB) 置位时，PRDH:PRD 的内容也被装入到 TIMH:TIM 中。

32位 CPU 周期寄存器 (TIMERxPRD)

TIMERxPRDH与之类似

15	14	13	12	11	10	9	8
TIF	TIE	保留	FREE	SOFT	保留		
R/W-0	R/W-0	R-0	R/W-0	R/W-0	R-0		
7	6	5	4	3			0
保留	TRB	TSS	保留				
R-0	R/W-0	R/W-0	R-0				

图例：R/W=读/写；R=只读；-n=复位后的值

TIMERxTCR 寄存器 (x=1, 2, 3)

TIMERxTCR 寄存器域描述

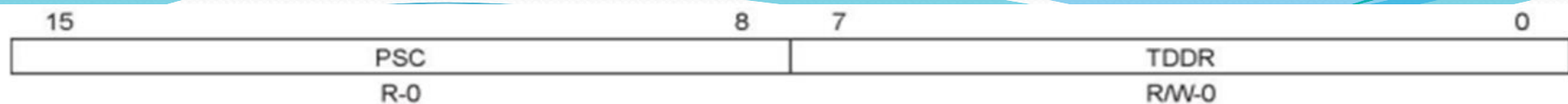
位	域	值	描述
15	TIF	0 1	CPU 定时器中断标志。 CPU 定时器还未递减到零。 写入 0 被忽略。 CPU 定时器递减到零时该标志置位。 通过写入 1 到这个位来清除相应的标志。
14	TIE	0 1	CPU 定时器中断使能。 CPU 定时器中断被禁能。 CPU 定时器中断被使能。如果定时器递减到零并且 TIE 置位，定时器就发出中断请求。
13-12	保留		保留

位	域	值	描述
11-10	FREE SOFT		CPU 定时器仿真模式：这两位是特殊的仿真位，当在高级语言调试器中碰到断点时，使用它们来确定定时器的状态。如果 FREE 位被置位，那么，在出现软件断点时，定时器继续运行（即，自由运行）。在这种情况下，SOFT 的值无关紧要。但是如果 FREE 为 0，SOFT 就生效。这时，如果 SOFT=0，定时器在下次 TIMH:TIM 递减时终止。如果 SOFT=1，定时器在 TIMH:TIM 已经递减到零时终止。
		FREE SOFT	CPU 定时器仿真模式
		0 0	在 TIMH:TIM 下次递减之后停止（硬停止）
		0 1	在 TIMH:TIM 递减到零之后停止（软停止）
		1 0	自由运行
9-6	保留	1 1	自由运行
			在软停止模式，定时器在关断之前产生一个中断（因为计数值到达 0 是产生中断的条件）。
9-6	保留		保留

TIMERxTCR寄存器描述_续

9-6	保留		保留
5	TRB	0 1	CPU 定时器重装位。 TRB 位读出时总为零。写 0 被忽略。 当写 1 到 TRB 时，TIMH:TIM 装入 PRDH:PRD 的值，并且，预分频器计数器 (PSCH:PSC) 装入定时器 divide-down 寄存器的值 (TDDRH:TDDR)。
4	TSS	0 1	CPU 定时器停止状态位。TSS 是一个 1 位的标志，用来停止或启动 CPU 定时器。 读出 0 表示 CPU 定时器正在运行。 将 TSS 设置为 0 来启动或重启 CPU 定时器。复位时，TSS 被清零，CPU 定时器立刻启动。 读出 1 表示 CPU 定时器被停止。 将 TSS 设置为 1 来停止 CPU 定时器。
3-0	保留		保留

TIMERxTCR寄存器描述_续



图例：R/W=读/写；R=只读；-n=复位后的值

TIMERxTPR 寄存器 (x=1, 2, 3)

TIMERxTPR 寄存器域描述

位	域	描述
15-8	PSC	CPU 定时器预分频计数器。这些保存定时器的当前预分频计数值。对于 PSCH:PSC 值大于 0 的每个定时器时钟源周期，PSCH:PSC 都减 1。在 PSCH:PSC 到达 0 之后的一个定时器时钟（定时器预分频器的输出）周期，PSCH:PSC 装入 TDDRH:TDDR 的内容，并且，定时器计数器寄存器（TIMH:TIM）减 1。只要软件置位定时器重装位（TRB），PSCH:PSC 也被重装。PSCH:PSC 可以通过读寄存器来检查，但它不能直接被设置。它必须从定时器 divide-down 寄存器（TDDRH:TDDR）获取值。复位时，PSCH:PSC 被设为 0。
7-0	TDDR	CPU 定时器 Divide-Down。每隔（TDDRH:TDDR+1）个定时器时钟源周期，定时器计数器寄存器（TIMH:TIM）就减 1。复位时，TDDRH:TDDR 位被清零。为了使整个定时器计数值增加一个整数因子，将该整数因子减 1 后写入 TDDRH:TDDR 位。当预分频器计数器（PSCH:PSC）值为 0 时，一个定时器时钟源周期之后，TDDRH:TDDR 的内容重新装入 PSCH:PSC，并且 TIMH:TIM 减 1。只要软件将定时器重装位（TRB）置位，TDDRH:TDDR 也重装 PSCH:PSC。

TIMERxTPR寄存器描述（TIMERxTPRH寄存器与之类似）

CPU定时器总结：

- 1) 有3个
- 2) 减计数
- 3) 定时长度其实是48位？
- 4) 每个可产生中断

CPU定时器使用要点：

- 1) 配置定时长度，包括PRD和TPR
- 2) 配置控制寄存器，包括是否允许中断、TRB和TSS

定时器主要功能：产生精确的定时事件

可以用中断处理
也可以用标志查询进行处理

定时器应用的流程：

- 1) 配置系统时钟
- 2) 初始化中断向量-----下一章的内容
- 3) 根据定时长度要求配置定时器相关寄存器
- 4) 编写中断服务程序，完成相应功能

举例：定时器0产生10kHz的定时中断

- 1) 系统时钟配置成60M
- 2) 周期寄存器配置成 $60\text{M}/10\text{k}=6000$

初始化程序:

```
void InitCpuTimer0()
{
    EALLOW;
    CpuTimer0Regs.TCR.bit.TSS = 1; // Stop CpuTimer0;
    CpuTimer0Regs.TCR.bit.TIF=1;
    CpuTimer0Regs.TIM.all=0;
    CpuTimer0Regs.PRD.all=6000-1; // 0.1mS
    CpuTimer0Regs.TPR.all=0;
    CpuTimer0Regs.TPRH.all=0;
    CpuTimer0Regs.TCR.bit.TRB=1;
    CpuTimer0Regs.TCR.bit.TIE=1;
    EDIS;
    CpuTimer0Regs.TCR.bit.TSS = 0; // Start CpuTimer0
}
```

● 5.7 EALLOW保护的寄存器

EALLOW保护某些控制寄存器，防止虚假的CPU写入。

下列模块中的寄存器受到EALLOW保护：

- 器件仿真寄存器；
- Flash寄存器；
- CSM寄存器；
- PIE向量表；
- 系统控制寄存器；
- GPIO MUX寄存器。

状态寄存器1（ST1）的EALLOW指示保护的状态

EALLOW位	CPU写	CPU读	JTAG写	JTAG读
0	忽略	允许	允许	允许
1	允许	允许	允许	允许

复位时，EALLOW位被清除，EALLOW保护被启用。
寄存器被保护时，CPU对受保护寄存器执行的所有写操作均被忽略，只允许对受保护寄存器执行CPU读、JTAG读和JTAG写。

如果通过执行**EALLOW**指令将EALLOW位置位，则允许CPU自由地写受保护寄存器。

更改寄存器之后，通过执行**EDIS**指令将EALLOW位清除来再次使寄存器被保护。

例5.6 EALLOW和EDIS的定义

```
#define EALLOW      asm(" EALLOW")  
#define EDIS        asm(" EDIS")
```

例5.7使用EALLOW和EDIS例子

```
//GPIO: GPxMUXn, GPxDIR
```

```
    EALLOW;
```

```
    GpioCtrlRegs.GPAMUX1.bit.GPIO0 = 0;
```

```
    GpioCtrlRegs.GPAMUX1.bit.GPIO1 = 0;
```

```
    GpioCtrlRegs.GPAMUX1.bit.GPIO2 = 0;
```

```
    GpioCtrlRegs.GPAMUX1.bit.GPIO3 = 0;
```

```
    GpioCtrlRegs.GPADIR.bit.GPIO0 = 1;
```

```
    GpioCtrlRegs.GPADIR.bit.GPIO1 = 1;
```

```
    GpioCtrlRegs.GPADIR.bit.GPIO2 = 1;
```

```
    GpioCtrlRegs.GPADIR.bit.GPIO3 = 1;
```

```
    EDIS;
```

v125\DSP2802x_headers\include

名称	修改日期	类型	大小
 DSP2802x_Adc.h	2015/4/23 21:17	C++ Header File	17 KB
 DSP2802x_BootVars.h	2015/4/23 21:17	C++ Header File	2 KB
 DSP2802x_Comp.h	2015/4/23 21:17	C++ Header File	3 KB
 DSP2802x_CpuTimers.h	2015/4/23 21:17	C++ Header File	6 KB
 DSP2802x_DevEmu.h	2015/4/23 21:17	C++ Header File	3 KB
 DSP2802x_Device.h	2015/4/23 21:17	C++ Header File	6 KB
 DSP2802x_ECap.h	2015/4/23 21:17	C++ Header File	6 KB
 DSP2802x_EPwm.h	2015/4/23 21:17	C++ Header File	22 KB
 DSP2802x_Gpio.h	2015/4/23 21:17	C++ Header File	12 KB
 DSP2802x_I2c.h	2015/4/23 21:17	C++ Header File	7 KB
 DSP2802x_NmiIntrupt.h	2015/4/23 21:17	C++ Header File	4 KB
 DSP2802x_PieCtrl.h	2015/4/23 21:17	C++ Header File	6 KB
 DSP2802x_PieVect.h	2015/4/23 21:17	C++ Header File	7 KB
 DSP2802x_Sci.h	2015/4/23 21:17	C++ Header File	8 KB
 DSP2802x_Spi.h	2015/4/23 21:17	C++ Header File	7 KB
 DSP2802x_SysCtrl.h	2015/4/23 21:17	C++ Header File	16 KB
 DSP2802x_XIntrupt.h	2015/4/23 21:17	C++ Header File	2 KB

v125\DSP2802x_common\source

☐ 名称	修改日期	类型	大小
 DSP2802x_Adc.c	2015/4/23 21:17	C Source	11 KB
 DSP2802x_CodeStartBranch.asm	2015/4/23 21:17	Assembler Source	4 KB
 DSP2802x_Comp.c	2015/4/23 21:17	C Source	4 KB
 DSP2802x_CpuTimers.c	2017/5/1 10:00	C Source	5 KB
 DSP2802x_CSMPasswords.asm	2015/4/23 21:17	Assembler Source	3 KB
 DSP2802x_DBGIER.asm	2015/4/23 21:17	Assembler Source	1 KB
 DSP2802x_DefaultIsr.c	2017/5/1 10:36	C Source	20 KB
 DSP2802x_DisInt.asm	2015/4/23 21:17	Assembler Source	2 KB
 DSP2802x_ECap.c	2015/4/23 21:17	C Source	3 KB
 DSP2802x_EPwm.c	2015/4/23 21:17	C Source	9 KB
 DSP2802x_Gpio.c	2015/4/23 21:17	C Source	3 KB
 DSP2802x_I2C.c	2015/4/23 21:17	C Source	4 KB
 DSP2802x_MemCopy.c	2015/4/23 21:17	C Source	2 KB
 DSP2802x_OscComp.c	2015/4/23 21:17	C Source	5 KB
 DSP2802x_PieCtrl.c	2015/4/23 21:17	C Source	3 KB
 DSP2802x_PieVect.c	2015/4/23 21:17	C Source	7 KB
 DSP2802x_Sci.c	2015/4/23 21:17	C Source	4 KB
 DSP2802x_Spi.c	2015/4/23 21:17	C Source	4 KB
 DSP2802x_SWPrioritizedDefaultIsr.c	2015/4/23 21:17	C Source	30 KB
 DSP2802x_SWPrioritizedPieVect.c	2015/4/23 21:17	C Source	9 KB
 DSP2802x_SysCtrl.c	2015/4/23 21:17	C Source	16 KB
 DSP2802x_TempSensorConv.c	2015/4/23 21:17	C Source	3 KB
 DSP2802x_usDelay.asm	2015/4/23 21:17	Assembler Source	3 KB

v125\DSP2802x_headers\include\DSP2802x_SysCtrl.h
v125\DSP2802x_common\source\DSP2802x_SysCtrl.c

时钟源、PLL
Watchdog
FLASH

v125\DSP2802x_headers\include\DSP2802x_CpuTimers.h
v125\DSP2802x_common\source\DSP2802x_CpuTimers.c

void ConfigCpuTimer(struct CPUTIMER_VARS *Timer, float Freq, float Period)